Lab 6

Deep Q-Network and Deep Deterministic Policy Gradient

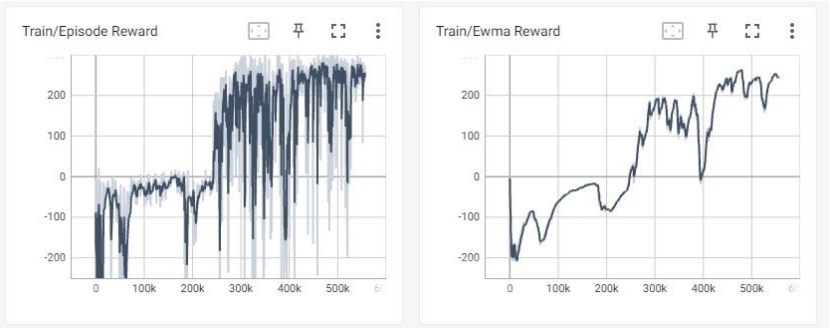
A. Experimental Results

DQN on LunarLander-v2 screenshot tensorboard and testing results

(1) testing results

```
PS C:\Users\tom89\OneDrive\桌面\Lab6> python dqn.py --test_only
C:\Users\tom89\AppData\Local\Programs\Python\Python310\lib\site-
  warnings.warn(colorize('%s: %s'%('WARN', msg % args), 'yellow'
Start Testing
episode 1: 227.56
episode 2: 275.40
episode 3: 258.86
episode 4: 261.81
episode 5: 299.51
episode 6: 254.55
episode 7: 298.61
episode 8: 285.08
episode 9: 303.93
episode 10: 284.60
Average Reward 274.99033053688345
```

(2) tensorboard

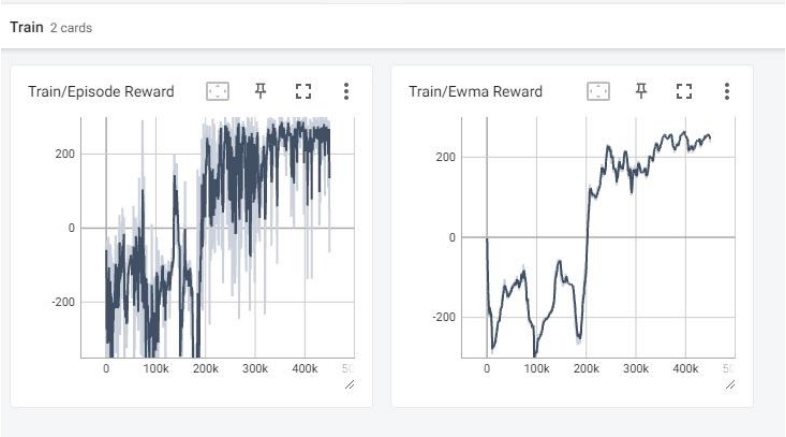


DDPG on LunarLanderContinuous-v2 tensorboard and testing results

(1) testing results

```
pp037@ec037:~/Lab6$ python3 ddpg.py --test_only
/home/pp037/.local/lib/python3.8/site-packages/gym/logger.py:30
  warnings.warn(colorize('%s: %s'%('WARN', msg % args), 'yellow
Start Testing
episode 1: 237.29
episode 2: 242.82
episode 3: 281.02
episode 4: 250.66
episode 5: 244.91
episode 6: 270.18
episode 7: 257.51
episode 8: 232.77
episode 9: 185.30
episode 10: 289.19
Average Reward 249.1656168411193
```

(2) tensorboard

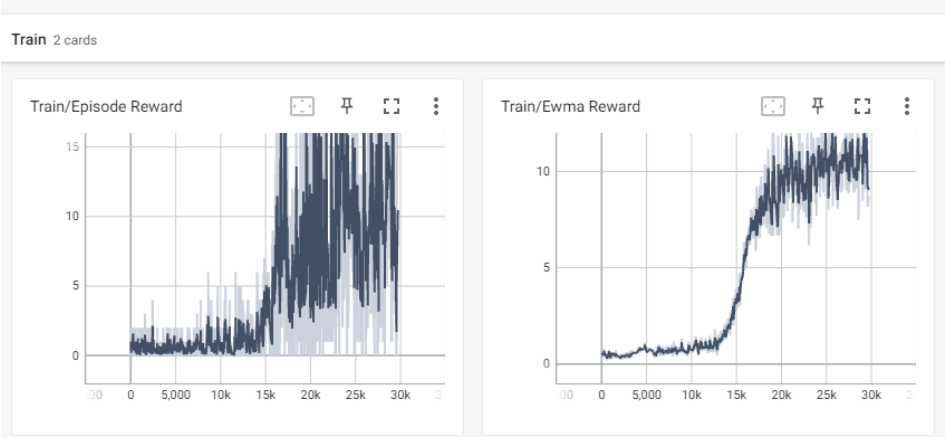


DQN_breakout on BreakoutNoFrameskip-v4 tensorboard and testing results

(1) testing results

```
Step: 4799565   Episode: 29646  Length: 318     Total
Start Testing
episode 1: 399.00
episode 2: 431.00
episode 3: 518.00
episode 4: 413.00
episode 5: 425.00
episode 6: 417.00
episode 7: 819.00
episode 8: 330.00
episode 9: 438.00
episode 10: 393.00
Average Reward: 458.30
```

(2) Tensorboard



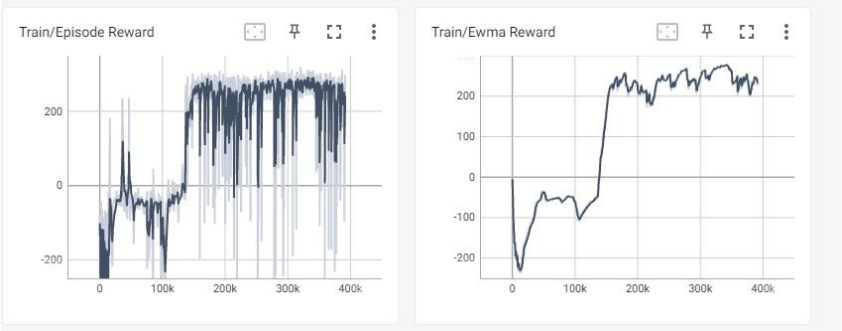
B. Experimental Results of bonus part(DDQN)

DDQN on LunarLander-v2 tensorboard and testing results

(1) testing results

```
PS C:\Users\tom89\OneDrive\桌面\Lab6> python ddqn.py --test_only
C:\Users\tom89\AppData\Local\Programs\Python\Python310\lib\site-pac
  warnings.warn(colorize('%s: %s'%('WARN', msg % args), 'yellow'))
Start Testing
episode 1: 238.93
episode 2: 283.64
episode 3: 263.85
episode 4: 271.45
episode 5: 307.74
episode 6: 239.50
episode 7: 292.54
episode 8: 294.45
episode 9: 307.28
episode 10: 232.92
Average Reward 273.2292281545359
```

(2) Tensorboard



C. Questions

- Describe your major implementation of both DQN and DDPG in detail

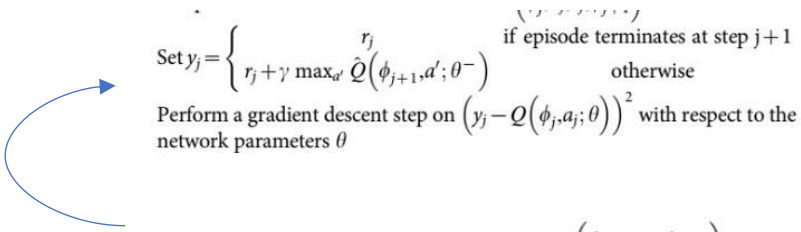
DQN

(1) Your implementation of Q network updating in DQN.
在 Q network 中，我們需要去更新兩個 network，分別是我們的 behavior network 和 target network，如以下二圖。

Update behavior network

```python
def _update_behavior_network(self, gamma):
    # sample a minibatch of transitions
    state, action, reward, next_state, done = self._memory.sample(
        self.batch_size, self.device)
    ## TODO ##
    # q_value = ?
    # with torch.no_grad():
    #     q_next = ?
    #     q_target = ?
    # criterion = ?
    # loss = criterion(q_value, q_target)
    q_value = self._behavior_net(state).gather(1, action.long())
    # print(action.long().shape)
    with torch.no_grad():
        q_next = self._target_net(next_state).max(1)[0].view(-1, 1)

        q_target = reward + gamma * q_next * (1 - done)
    criterion = nn.MSELoss()
    loss = criterion(q_value, q_target)
    # raise NotImplementedError
    # optimize
    self._optimizer.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(self._behavior_net.parameters(), 5)
    self._optimizer.step()
```



利用這個公式，利用 replay memory 中的 $\left(\phi_j, a_j, r_j, \phi_{j+1}\right)$，配合計算出目前的 q_value，並且計算出 target net 的 q_target，求出兩者間的 MSE 當作 loss 來更新 model。

Update target network

```
def _update_target_network(self):
    '''update target network by copying from behavior network'''
    ## TODO ##
    self._target_net.load_state_dict(self._behavior_net.state_dict())
```

$$\text{Every } C \text{ steps reset } \hat{Q} = Q$$

根據 algo 中的這項，每走 100 steps 就做一次更新 target net 成目前的 behavior net。

(2) Select action in DQN.

select action

```
def select_action(self, state, epsilon, action_space):
    '''epsilon-greedy based on behavior network'''
    ## TODO ##
    if np.random.uniform() < epsilon:
        action = action_space.sample()
    else:
        state = torch.unsqueeze(
            torch.FloatTensor(state), 0).to(self.device)
        with torch.no_grad():
            actions_value = self._behavior_net(state)
            action = torch.max(actions_value, 1)[1].cpu().data.numpy()
            action = action[0]

    return action
    # raise NotImplementedError
```

With probability ε select a random action a_t
otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$

利用 algo 中的這個公式，決定接下來的 action。當 epsilon 小於 random number 時，隨機選個 action，大於 random number 時，利用目前的 state 求出最大的 actions_value 來決定 action。

DDPG

(3) Select action in DDPG.

select action

```python
def select_action(self, state, noise=True):
    '''based on the behavior (actor) network and exploration noise'''
    ## TODO ##
    state = torch.unsqueeze(torch.FloatTensor(state), 0).to(self.device)

    with torch.no_grad():
        if noise:
            noise_action = torch.tensor(
                self._action_noise.sample(), device=self.device)
            action = self._actor_net(state.view(
                1, -1)) + noise_action.view(1, -1)
        else:
            action = self._actor_net(state.view(1, -1))

    return action.cpu().numpy().squeeze()
    # raise NotImplementedError
```

Select action $a_t = \mu(s_t|\theta^\mu) + N_t$ according to the current policy and exploration noise

利用 algo 中的這個公式，來將 actor net 中的 state 取出，並在 training 時加上 noise，來達到 exploration。

(4) Your implementation and the gradient of actor updating in DDPG

Update actor

```python
## update actor ##
# actor loss
## TODO ##
action = actor_net(state)
actor_loss = -critic_net(state, action).mean()
```

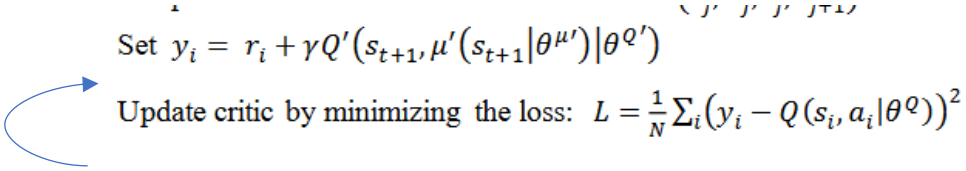
$$\nabla_{\theta^\mu}\mu|s_i \approx \frac{1}{N}\sum_i \nabla_a Q(s,a|\theta^Q)|_{s=s_i,a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|s_i$$

利用 algo 中的這個公式，取得目前的 actor net 中的 state action，然後與 critic net 一起計算其 actor loss。而且他的更新方式是用 gradient ascent，而這與 gradient descent 相反，所以要在 loss 加個負號。

(5) Your implementation and the gradient of critic updating in DDPG

Update critic

```
## update critic ##
# critic loss
## TODO ##
q_value = critic_net(state, action)
with torch.no_grad():
    a_next = target_actor_net(next_state)
    q_next = target_critic_net(next_state, a_next)
    q_target = reward + ((1-done) * gamma * q_next).detach()
criterion = nn.MSELoss()
critic_loss = criterion(q_value, q_target)
```

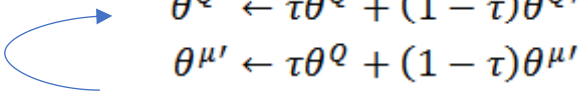


利用 algo 中的這個公式，這邊的更新原理跟 DQN 那邊其實差不多，只是在更新 q_target 時，要用上 target net。

(6) Update target network in DDPG

Update target network

```
def _update_target_network(target_net, net, tau):
    '''update target network by _soft_ copying from behavior network'''
    for target, behavior in zip(target_net.parameters(), net.parameters()):
        ## TODO ##
        target.data.copy_(tau * behavior.data + (1.0 - tau) * target.data)
```



利用 algo 中的這個公式，這個方法是不希望讓兩個 net 突然跳得太遠的感覺，可以更順暢更新。

- Explain effects of the discount factor.
  discount factor 的值會去影響 model 的發展，越大會對未來的 reward 更看重，反之則看中當下。而設置太大會使 model 很不穩定，而且會變慢，設置太小又會忽視未來的可行性，所以 discount factor 是用來當作一個平衡的效果，平衡未來及當下的比例。

- Explain benefits of epsilon-greedy in comparison to greedy action selection.
  這個原因就是因為我們的 model 是需要 exploration 的，也就是我們不能一直探索目前最好的 action，有時候應該要 random action 一下，選擇一個可能當下不是最好的結果，但他有可能會是對未來有相當大貢獻的一步，所以我們會需要使用 epsilon-greedy。

- Explain the necessity of the target network.
  有助於解決所謂的 Overestimation 的問題，使用單一個 net 的話會有高估的可能，所以加入一個 target network，來幫助我們更好的 learning 和 convergence 到最佳解。

- Describe the tricks you used in Breakout and their effects, and how they differ from those used in LunarLander.
  神經網路的差異性，因為在 Breakout 中我們是利用素像圖像當輸入，所以會使用 CNN 來提取特徵，而在 LunarLander-v2 中，我們是連續的狀態空間，所以更多是使用全聯接層(nn.linear())來處理這些特徵。

  但最有感的還是在時間問題，這個 Breakout 非常花時間在訓練，而大部分的 code 在 LunarLander-v2 與 Breakout 中是一樣的，而有一個比較有趣的是我在 Breakout 時，把滿多原本在 LunarLander-v2 是用 tensor 形式的 data 轉成 numpy 的形式，這是為了加快我們的訓練時間 (我原本沒注意到，用了 3060ti 跑 20 個小時才大概 400w steps，而 test 分數才 340 左右，後來受不了忍心 shutdown，重跑後，不到半天就 4xx 了)。